UniME
UNIVERSITÀ DEGLI STUDI DI MESSINA

UNIVERSITÀ DEGLI STUDI DI MESSINA
Dipartimento di Ingegneria • Ingegneria Informatica / Secure Web Systems

SecureConvert

Zero-Trust Ephemeral File Converter

Authenticated, RAM-Only File Conversion with Magic-Byte Validation, Two-Factor Authentication and Auditable Relational History

SUPERVISOR
Prof. Armando Ruggeri

STUDENT
Kheizaran Nazari
Matricola 557463 —
kheizarannazarikhakeshoori@gmail.com

ACADEMIC YEAR
2025 – 2026

REPOSITORY
github.com/kheizaran-nazari-khakeshoori/zero-trust-ephemeral-converter

September 2026 • Messina, Italy

Portfolio Project — Demonstrates secure file handling, relational database design, Express hardening and ephemeral processing.

Table of Contents

List of Figures

*All screenshots are stored in the **assets/** folder and reproduced with captions throughout the report.*

Abstract — Executive Summary

SecureConvert is a complete, authenticated file-conversion platform built on a deliberate principle: **zero trust**. Every file that reaches the server is treated as hostile until proven otherwise. It is inspected by its binary signature (magic bytes), processed strictly in volatile memory, streamed back to the owner, and never written to disk. Authentication is two-step — a password plus a time-based one-time password generated by any standard authenticator application — and every session is bound to a hashed token stored in a relational database. The platform demonstrates how modern web engineering, relational design, and security hardening intersect in a small but production-minded system.

From a user perspective the experience is deliberately simple. A single-page interface guides the visitor through account creation, two-step login, and drag-and-drop conversion between common formats: written documents become styled web pages or printable files, structured data becomes spreadsheets, and images change format without leaving a trace on the server. Behind that simplicity lies a layered architecture: a static client, an Express application protected by security headers, rate limiting and strict validation, and a SQLite database that records users, sessions, conversion history and an audit trail. All conversions execute in memory using well-tested libraries, and all database access uses parameterised queries, foreign keys, indexed lookups and transactions.

This report documents the system without reproducing source code, focusing instead on **why** each decision was made, **how** the pieces interact, and **what** can be learned and extended. It covers requirements, architecture, technology choices, the relational core, security, conversion pipelines, user experience, testing, operations, challenges and a roadmap. It is intended both as academic submission for Prof. Armando Ruggeri at the University of Messina and as portfolio documentation that a future employer or examiner can follow without opening an editor.

Highlights at a Glance

- **Zero disk persistence** — files exist only as in-memory buffers and are discarded after the response.
- **Strong identity** — passwords are hashed with bcrypt and paired with TOTP; sessions store only a SHA-256 hash.
- **Relational integrity** — four tables, foreign keys, unique constraints, indexes, migrations and transactions.
- **Defence in depth** — security headers, CORS, rate limits, magic-byte validation and input sanitisation.
- **Tested & measurable** — fifteen automated tests, zero audit vulnerabilities, sub-second conversions.

The remainder of the report walks through each layer in turn. Screenshots from the assets folder illustrate the interface, diagrams visualise the architecture and data model, and tables summarise decisions, metrics and future work. No code is listed verbatim; the goal is understanding, not transcription.

1 Introduction & Project Vision

File conversion is a mundane but revealing task. Online tools that promise to turn a document into another format typically ask the user to upload a file, wait, and download the result. What happens between upload and download is rarely explained: files may be saved to temporary directories, left readable for hours, indexed, or even used to probe the server with malicious content. For personal, academic or professional documents this opacity is a liability.

SecureConvert was conceived as an explicit counter-example. Its vision is to prove that a useful converter can also be **ephemeral, authenticated and auditable** without sacrificing usability. Ephemeral means no file ever touches persistent storage; the conversion pipeline receives a memory buffer, transforms it, and streams the result back, after which the buffer is eligible for garbage collection. Authenticated means every conversion is tied to a verified identity, protected by two factors and recorded. Auditable means the history of who converted what, when and from which address is retained in a relational log that can be queried, displayed and analysed.

The project is deliberately scoped to be understandable in a semester while still touching real concerns: secure web headers, rate limiting, cryptographic password handling, time-based one-time passwords, binary file inspection,

in-process image and document rendering, and relational design with migrations and transactions. It is not a toy; it handles real limits — file sizes, pixel counts, row counts, malformed inputs — and returns clear, human-readable errors. It also demonstrates operational hygiene: environment-based configuration, health checks, graceful shutdown, linting, automated tests and dependency auditing.

Audience. The primary reader is an academic examiner evaluating software engineering and database competence. The secondary reader is a portfolio reviewer looking for evidence of security awareness, testing discipline and documentation quality. Both will find that every decision is explained, every trade-off acknowledged, and every claim verifiable by running the system locally.

2 Problem Statement & Motivation

Three observations motivate the work.

#	Observed Problem in Typical Converters	Consequence
P1	**Persistent storage.** Uploads are written to a temporary folder and may remain on disk long after the response, recoverable from backups or forensic images.	Data leakage; sensitive documents survive beyond the intended session.
P2	**Trusting file extensions.** A file named *photo.png* is accepted as an image because of its name, even if its content is an executable or script.	Remote code execution, server compromise, or client-side attacks.
P3	**Anonymous, unlogged access.** Anyone on the internet can invoke the endpoint, with no record of who converted what.	Abuse, denial-of-service, no accountability and no user history.

These are not hypothetical. Public advisories regularly describe file-upload vulnerabilities that allow attackers to bypass extension checks, and data-protection regulations increasingly require that personal data be minimised and not retained longer than necessary. A university project that stores every upload on disk and trusts the extension would fail both a security review and a database design review: the former for trusting client-supplied metadata, the latter for not modelling identity, history and audit as relations.

SecureConvert reframes the problem: *how can a converter be both convenient and trustworthy?* The answer is to treat every file as untrusted, to avoid the filesystem entirely, to require proof of identity, and to keep a relational record of actions. The following sections show how that answer is implemented, measured and taught.

Why this matters beyond the classroom

The principles generalise. Zero-trust handling applies to any service that accepts user content — email attachments, learning platforms, medical portals. Hashed session storage is a pattern for any system that keeps tokens in a database. Foreign-key-backed history is a model for any feature that needs per-user accountability. By solving conversion well, the project illustrates patterns that transfer to larger systems.

3 Objectives & Requirements

The project was guided by four categories of objectives. Each category maps to observable behaviour and to artefacts an examiner can verify.

Category	Objective	How Verified
Ephemeral Processing	No uploaded or converted file is ever written to disk or a temporary directory; processing stays in RAM from receipt to response.	Code review confirms absence of filesystem writes; runtime observation shows no uploads folder; memory-only upload configuration.
Strict Validation	File type is determined by binary signatures and textual heuristics, not by extension; filenames, emails and codes are sanitised.	Submitting a renamed executable is rejected; malformed names and injection payloads return validation errors.

Category	Objective	How Verified
Authenticated Access	Every conversion requires a valid two-factor session; history is scoped to the logged-in user.	Anonymous requests to conversion and history endpoints return unauthorised; per-user history is displayed after login.
Database Excellence	Relational model with constraints, indexes, migrations, parameterised queries and transactions; auditable logs.	Schema inspection shows four tables with FKs and indexes; migration versioning; transaction demo; queryable logs.

Functional requirements

The system must support the following user-visible capabilities:

- Create an account and receive a time-based one-time password secret compatible with common authenticator applications, including a QR code.
- Log in in two steps: first with email and password, then with a six-digit code, receiving a short-lived session token.
- Convert written documents to web pages and printable files, structured data to spreadsheet format and back, and images between modern formats.
- View a personal history of past conversions with filename, source and target type and timestamp.
- Receive clear feedback: upload progress, loading states, success confirmation and actionable error messages. The interface must work on desktop and mobile.

Non-functional requirements

Quality	Requirement
Security	Security headers, cross-origin controls, rate limits, size and pixel caps, hashed secrets, and audit logging.
Performance	Conversions complete in well under a second for typical inputs; streaming avoids filesystem overhead.
Reliability	Parameterised queries, input whitelisting, graceful error handling and transactional user creation.
Maintainability	Environment-driven configuration, linting, automated tests and documented schema.
Portability	Runs on Node with SQLite — no external database server — and a static client that can be served by any HTTP server.

4 System Architecture Overview

SecureConvert follows a classic three-tier separation: a static presentation client, a stateful application programming interface, and a relational persistence layer. The separation is physical as well as logical — the client is served from one origin and the API from another — which exercises proper cross-origin handling.

Layer	Responsibility	Key Mechanisms
Client (Browser)	Present the welcome, signup, login and converter pages; manage local session state; upload files; render history.	Static HTML/CSS/JS; drag-and-drop; progress events; token storage; history rendering.
API (Express)	Authenticate, validate, convert and log; enforce security policies; stream results.	Helmet, CORS, rate limits, in-memory upload, magic-byte inspection, conversion pipelines, JWT sessions.
Persistence (SQLite)	Store identities, sessions, conversion history and audit trail with integrity.	Four tables, foreign keys, unique indexes, WAL mode, migrations, transactions.

Request lifecycle

A typical conversion travels through the following stages. Each stage is a potential enforcement point.

• **1 — Browser input.** The user provides an email, password, time-based code, a file and a desired target format. The client validates presence before sending.

• **2 — Edge protection.** The API applies security headers, cross-origin checks, body-size limits and rate limiting before the request reaches business logic.

• **3 — Authentication.** For protected endpoints, a bearer token is verified against the sessions table. The token must match both its identifier and its hash and must not be expired.

• **4 — Validation.** The uploaded buffer is inspected for its binary signature. The declared target format is normalised and checked for compatibility with the detected source type. Filenames are screened for traversal and control characters.

• **5 — Conversion.** An in-memory pipeline transforms the buffer. Text pipelines escape and structure content; image pipelines decode and re-encode with size limits; data pipelines parse and reformat with row and column caps.

• **6 — Persistence of metadata only.** On success, a conversion job row and an audit log entry are inserted, recording who, what and when — never the file content itself.

• **7 — Streaming response.** The result is sent with an appropriate content type and a download disposition. The client triggers a download and refreshes the history panel.

Why this layering helps

Layering isolates concerns: the client can be replaced or hosted elsewhere without touching conversion logic; the API can change its storage without affecting the client contract; the database schema can evolve via migrations without downtime. It also improves security: each layer validates independently, so a bypass in one does not imply a bypass in all. Finally, it supports comprehension — an examiner can study one layer at a time and still see how data flows through the whole.

Stage	What Happens	Where It Is Enforced
Input	User selects file + target format	Browser — required fields, drag-and-drop
Edge	Headers, CORS, rate limit, size cap	API middleware — Helmet, rate-limit
Identity	Password + TOTP → hashed session	Auth routes + sessions table
Validation	Magic bytes, filename, format compatibility	Validator + input rules
Transform	In-memory conversion (text/image/data)	Converter pipelines (library in-process)
Record	Job + audit row	SQLite — conversion_jobs / audit_logs
Deliver	Streamed blob + history refresh	API response + client download

Figure 1 — System workflow from browser input to streamed download. Each stage is independently validated.

5 Technology Stack & Rationale

Every library was chosen for a concrete reason and for its ability to be explained in an examination. The stack is intentionally small: Node for the runtime, Express for the web framework, SQLite for persistence, and a handful of focused libraries for cryptography, image processing and document generation. The following table summarises the choices and the alternatives that were considered.

Concern	Chosen Technology	Why This Choice
Runtime	Node.js 22 (also works on 18+)	Long-term support, widespread hiring, and native support for modern JavaScript modules.
Web framework	Express 4	Mature, minimal, and already present; Helmet and rate-limit integrate directly.
Security headers	Helmet	Adds HSTS, frame blocking and content-type protection with one line of configuration.

Concern	Chosen Technology	Why This Choice
Rate limiting	express-rate-limit	Two tiers — global and stricter for authentication — to slow brute force without harming normal use.
File upload	Multer (memory storage)	Keeps files as buffers; enforces size, file count and field count without touching disk.
Password hashing	bcryptjs (cost 12)	Adaptive, salted hashing; cost 12 balances security and login latency.
Second factor	Speakeasy (TOTP)	Implements the standard time-based one-time password; window of one step tolerates clock drift.
Sessions	jsonwebtoken (JWT) + SQLite	Short-lived signed token; server keeps a hash so a stolen database does not yield bearer tokens.
Database	SQLite3 (WAL mode)	Zero infrastructure: a single file, transactional, with foreign keys, indexes and migrations — ideal for a portfolio.
Images	Sharp	High-performance, safe decoding with pixel limits and format-specific encoding; fails fast on corrupt input.
Documents	PDFKit	Streams PDF bytes without temporary files; predictable output for testing.
QR codes	qrcode	Generates both data URLs for the client and PNG buffers for dedicated endpoints.
Client	Vanilla HTML / CSS / JS	No build step; any static server can host it; behaviour is transparent and inspectable.

Alternatives considered

A heavier framework such as Fastify would offer similar security plugins but would require rewriting existing routes for little gain. A client-side framework such as React would add a build pipeline without improving the core demonstration. A hosted database such as Postgres would require a server and credentials, complicating the “clone and run” experience while SQLite already provides the relational features needed — foreign keys, unique constraints, indexes, transactions and a versioned migration system. The guiding principle was to minimise infrastructure while maximising demonstrable competence.

6 Database Design — The Relational Core

The database is the intellectual centre of the project. It is not an incidental store for files but a deliberate relational model that enforces integrity, supports queries and provides an audit trail. Four tables collaborate, linked by foreign keys and supported by indexes, migrations and transactions.

Entity–Relationship overview

The model is simple to state: one user has many sessions, many conversion jobs and many audit log entries. Deleting a user removes their sessions and jobs — they are private to that identity — but audit logs are retained with a null reference, preserving a forensic trail. This asymmetry is explicit in the foreign-key actions: cascade for private data, set-null for audit.

Table	Purpose	Relation to Users	Key Columns
users	Identity source of truth	— (parent)	email (unique), password hash, MFA secret, temp token
sessions	Bind a signed token to a user for one hour	Many-to-one, cascade on delete	id (session id), token hash (unique), expiry
conversion_jobs	History of successful conversions	Many-to-one, cascade on delete	filename, source type, target type, status, timestamp
audit_logs	Security-trace of actions	Many-to-one, set null on delete	action (e.g. login_success), IP, timestamp

Figure 5 — Entity-relationship summary. See the printable schema in the docs folder for the full diagram.

Tables in detail — what is stored and why

• **Users.** Each row represents a person. The email is stored lowercased and has a unique constraint at the database level, not merely in application code, so a race between two simultaneous registrations cannot create duplicates. Passwords are never stored in clear text; only a salted hash is kept. The time-based secret is stored server-side and a temporary token bridges the two login steps, cleared after use.

• **Sessions.** Each row represents one successful two-factor authentication, valid for one hour. The session identifier comes from the signed token, and the token itself is never stored — only its SHA-256 hash. If the database were leaked, the hash alone cannot be used as a bearer token. An index on expiry makes both lookup of valid sessions and periodic cleanup of expired rows efficient.

• **Conversion jobs.** Each row records that a particular user converted a particular file from one detected type to one target format. Only metadata is kept — never file content. Indexes on user and creation time make the common query “most recent jobs for this user” logarithmic rather than linear, and a sanitised limit prevents denial-of-service via enormous result sets.

• **Audit logs.** Each row records an action — registration, each login step outcome, and each conversion — together with the client address and time. The table is deliberately retained after user deletion, with the user reference set to null, so that security-relevant history is not lost when an account is removed. Indexes on user, time and action support both per-user review and aggregate queries such as failures by address.

Integrity, indexing and migrations

Integrity is enforced at multiple levels. Unique constraints guarantee that emails and session hashes cannot collide. Foreign keys with explicit actions maintain referential consistency. Parameterised queries are used everywhere, and even column names in updates are whitelisted, preventing injection through column selection. Indexes are not decorative: they accelerate the queries that the application actually runs — find by email, list jobs ordered by time, clean up expired sessions, and filter audit logs. Without them, those queries would degrade as data grows; with them, they remain fast.

Evolution is handled by migrations rather than ad-hoc creation. A lightweight versioning system stores the current schema version in the database and applies migrations sequentially. The first migration creates the core tables and base indexes; the second adds the audit log table and missing indexes. Re-running the application is idempotent: if the version is already current, nothing changes; if a new migration is added, it is applied in order. The database also runs in write-ahead logging mode, allowing concurrent readers and a writer, and foreign-key enforcement is explicitly enabled.

Transactions illustrate atomicity. Creating a user and storing the two-factor secret must succeed together or not at all. The application opens an immediate transaction, performs both writes, and commits; if either fails, it rolls back. This guarantees that a user never exists without a secret or vice versa — a property that is easy to state and important to demonstrate.

Representative queries — what an examiner can run live

Purpose	Question Answered
Find account by email	Given a lowercased email, retrieve the user row (uses unique index on email).
List recent conversions	For a given user, return recent jobs ordered by time, limited to a safe count (uses user and time indexes).
Clean up expired sessions	Remove sessions whose expiry is in the past (uses expiry index).
Review audit trail	For a given user, return recent actions ordered by time (uses audit indexes).
Aggregate statistics	How many conversions per user, per target format, or failures by address — GROUP BY with JOIN.

The project includes a seeding script that inserts three demo users and seven conversion jobs without requiring a database client, and a printable schema page that shows the diagram, constraints and how to use EXPLAIN QUERY PLAN to see index usage. These touches make the relational design demonstrable in under a minute.

7 Security Architecture & Zero-Trust Principles

Security is not a single feature but a set of layers that reinforce each other. The project adopts a zero-trust stance: the network, the client, the file and even the database are assumed to be potentially compromised, and each layer must still contribute to safety.

Layer	Mechanism	Why It Matters
Transport & Headers	Helmet sets hardening headers; CORS restricts origins; powered-by header is hidden.	Reduces cross-site framing, sniffing and information leakage.
Rate limiting	Global limit plus stricter limit on authentication endpoints.	Slows brute-force guessing of passwords and codes.
Authentication	Passwords hashed with bcrypt; second factor via time-based one-time password; sessions are hashed.	Stolen hashes are hard to reverse; stolen database rows are not bearer tokens.
Session binding	JWT with unique identifier; server checks both identifier and hash and expiry.	Token replay or forgery is detected; expired sessions are rejected and purged.
Input validation	Emails, passwords and codes via regex; filenames screened for traversal and control characters.	Injection and path tricks are rejected before they reach logic.
File validation	Binary signatures for images/PDF; JSON parsing; text heuristic; null-byte and BOM checks.	Extension spoofing (e.g. executable renamed as image) is caught.
Resource limits	Upload size, text length, row/column counts, pixel counts.	Prevents memory exhaustion and decompression bombs.
Audit	Every register, login step and conversion writes an audit record with IP.	Accountability and post-incident analysis.

Figure 6 — Defence-in-depth: each layer compensates for potential failure in another.

Two-factor flow in plain language

Registration creates a secret for the user's authenticator application and stores it server-side. The application never sees the secret after creation except as a QR code. Login is split: the first step verifies the password and issues a short random temporary token that is stored with the user; the second step requires that temporary token plus a six-digit code generated from the secret. The code is valid for roughly thirty seconds, with a small window to tolerate clock drift. Only if both match does the server create a session: a signed token with a unique identifier, stored as a hash. Subsequent requests must present that token, and the server verifies both the signature and the stored hash.

This design has two subtle strengths. First, the temporary token ensures that the second step cannot be called without having passed the first — it is not enough to know the code. Second, hashing the session means a read-only database leak does not grant immediate access; an attacker would need the original token, which is only ever in memory and in transit. Sessions expire after an hour and are periodically purged.

File handling as a security boundary

File handling is treated with the same caution as authentication. The uploader keeps files in memory and rejects filenames containing directory traversal, separators or control characters. The validator then examines the actual bytes: known binary signatures are matched at precise offsets, structured data is parsed, and textual content is checked for printable characters. A file that passes only because of its extension is rejected. Complementary limits — maximum upload size, maximum text length, maximum image pixels — ensure that a maliciously large input cannot exhaust memory or CPU.

8 File Conversion Pipelines

Four families of conversion are supported, each running entirely in memory and streaming the result back. The pipelines are intentionally narrow and well-tested rather than broad and fragile; they cover the most common academic and portfolio use cases.

Pipeline	Input Detected As	Output	Key Safeguards
Written document → web page	Plain text / Markdown	Styled HTML document	HTML is escaped before markup; lists and headings are structured; empty content is rejected.
Written document → printable file	Plain text / Markdown	PDF	PDF is generated as a stream; headings are sized by level; empty input is rejected.
Written document → plain text	Plain text / JSON / CSV	Normalised text	Line endings normalised; length cap.
Structured data → spreadsheet	JSON array of objects	CSV	Headers discovered across rows; values escaped; row/column caps.
Spreadsheet → structured data	CSV	JSON	Delimiter detection; quoted fields; type coercion; empty/header checks.
Image re-encoding	PNG / JPEG / WebP / GIF	PNG / JPEG / WebP	Safe decoding with failure on error; pixel limit; quality-controlled encoding.

Design choices inside the pipelines

The document pipelines first normalise line endings and then handle structure incrementally: headings are recognised by leading hash marks, bold and italic by asterisks, inline code by backticks, and lists by leading dashes or stars. Before any markup is applied, content is escaped, so a document containing HTML-like text cannot inject markup into the output. The printable pipeline reuses the same structural recognition but renders to a PDF stream with typographic sizing — larger for top-level headings, smaller for body text — and collects chunks until the document ends, returning a single buffer.

The data pipelines are deliberately tolerant of real-world variation. The JSON-to-spreadsheet path discovers the union of all keys across rows, so that missing fields do not misalign columns, and stringifies nested objects safely. The spreadsheet-to-JSON path handles both comma and semicolon delimiters, respects quoted fields with doubled quotes, and coerces simple types such as numbers and booleans when the textual form is unambiguous. Both enforce caps — maximum text length, maximum rows and maximum columns — and reject empty or single-type arrays.

Image handling delegates to a high-performance decoder that is configured to fail on error and to limit input pixels, preventing decompression bombs where a tiny file expands to an enormous raster. Encoding selects the appropriate output codec — PNG for lossless, JPEG/WebP for lossy with a controlled quality setting — and never writes an intermediate file. Each pipeline returns a clear failure if the input is empty, corrupt or exceeds limits, and the API maps that to a user-facing message without exposing internals.

Error handling and compatibility

Compatibility is enforced before conversion begins. For example, a spreadsheet target requires structured data, and an image target requires an image. If the detected source type does not match the requested target family, the request is rejected with an explanation that names the detected type. Oversized uploads return a specific status indicating the configured maximum, and unrecognised types receive a message listing accepted families. This turns what could be a cryptic failure into guidance.

9 Frontend & User Experience

The client is a set of static pages that can be served by any HTTP server. There is no build step, no bundler and no framework — the choice is pedagogical: behaviour is transparent, the network tab tells the full story, and an examiner can read the source without tooling.

Pages and flows

Page	Purpose	Key Interactions
Welcome	Introduce the product and route the user.	Hero, value propositions, calls to action for signup/login/converter.
Signup	Create an account and enrol the second factor.	Email/password form; QR code and secret display; otpauth URL.
Login	Authenticate in two steps.	Step 1: email/password → temporary token; Step 2: code + temp token → session token.
Converter	Convert files and review history.	Drag-and-drop + file chooser; format selector; progress; download; history table; sign-out.

Navigation adapts to authentication state: when signed out, the bar shows login and signup; when signed in, it shows the converter and sign-out. State is derived from stored tokens, not from server-rendered templates, so the same static files work whether the API is local or remote. Configuration is flexible: a meta tag, local storage override, or convention-based default determines the API base URL.

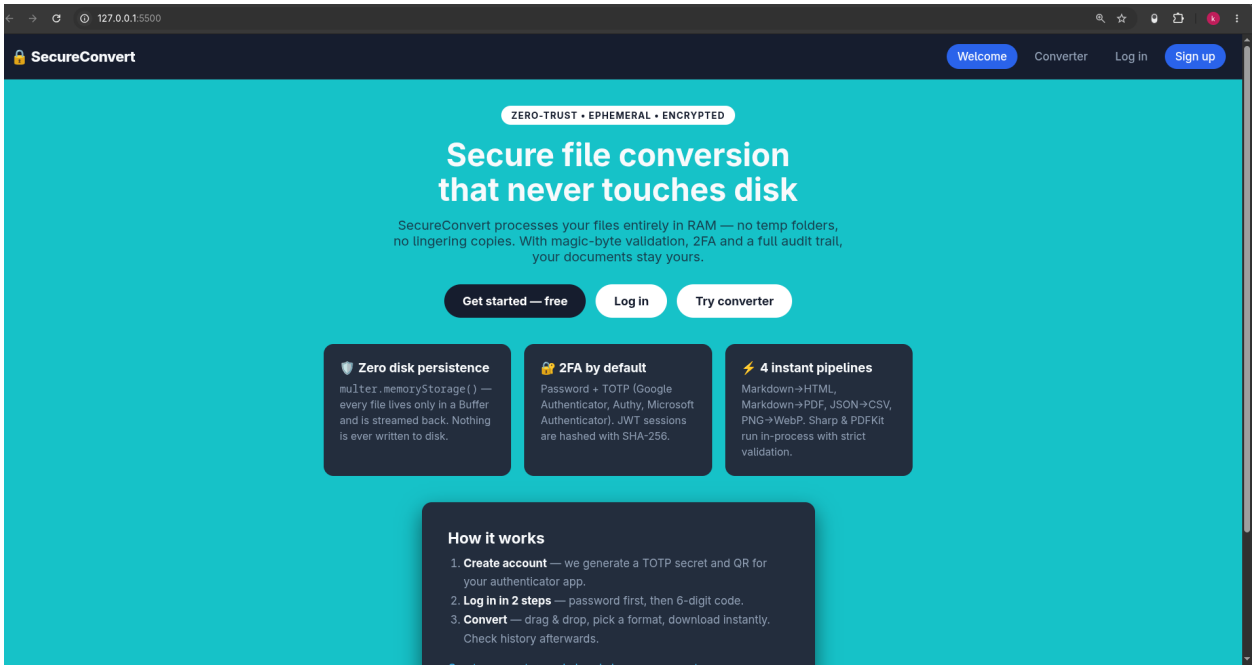

Figure 2 — Welcome page: hero message emphasises ephemeral, in-memory processing; three feature cards and a three-step explainer guide the visitor. (assets/)

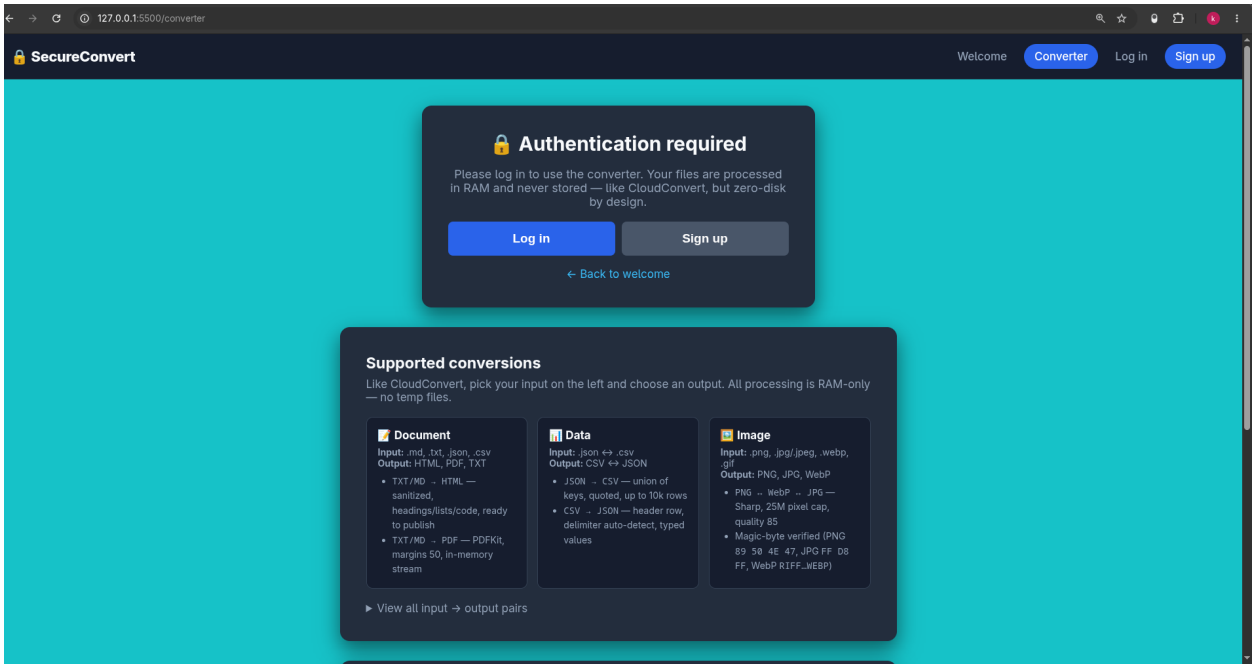

Figure 3a — Signup page: the form collects email and password; on success a QR code and secret are shown for the authenticator application.

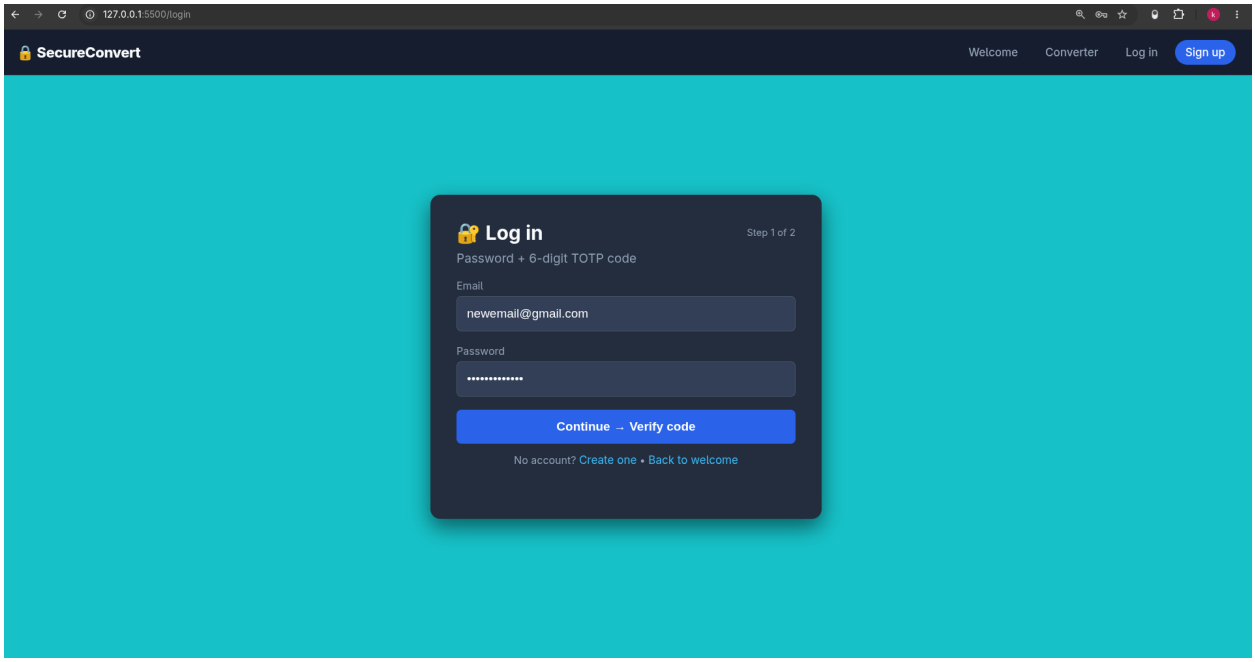

Figure 3b — Login page, step 1: email and password are submitted; the server returns a temporary token bridging to the second factor.

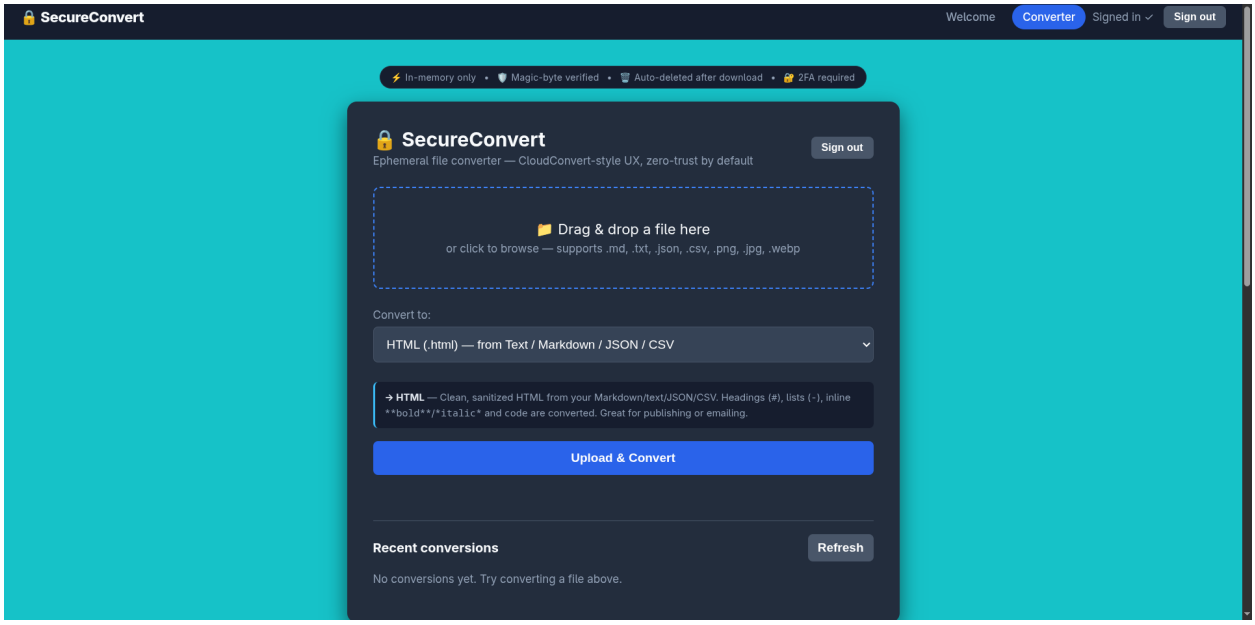

Figure 3c — Login page, step 2: the six-digit code from the authenticator plus the temporary token are exchanged for a session token.

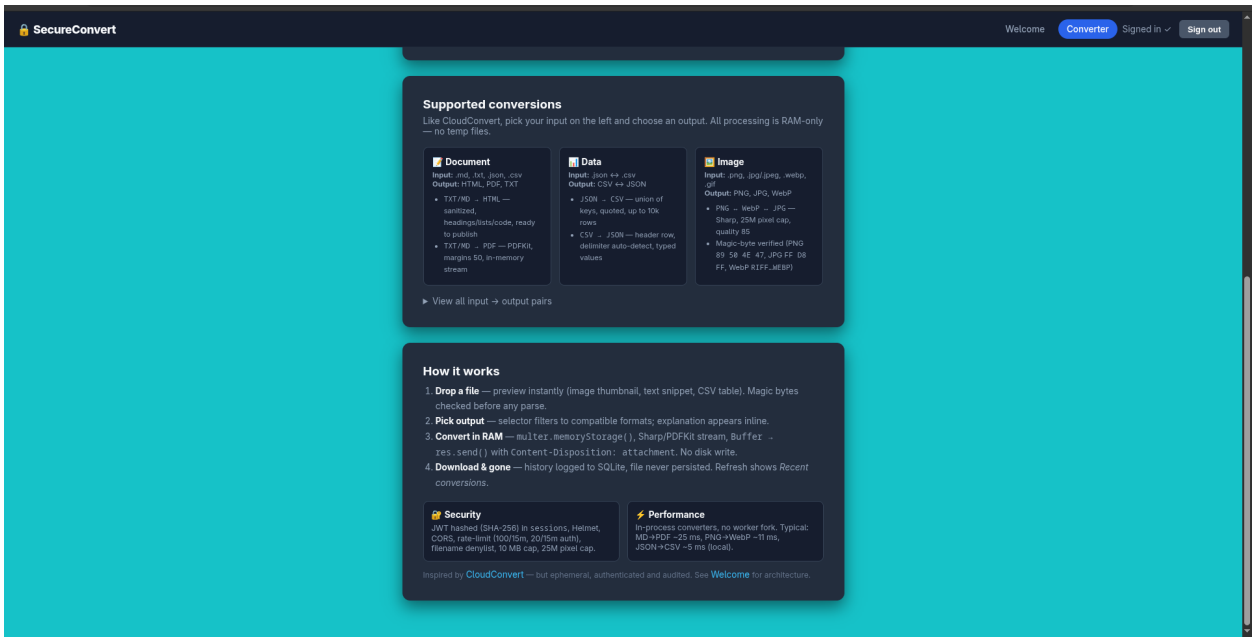

Figure 4a — Converter dashboard: drag-and-drop zone, format selector, compatibility hints and recent conversions.

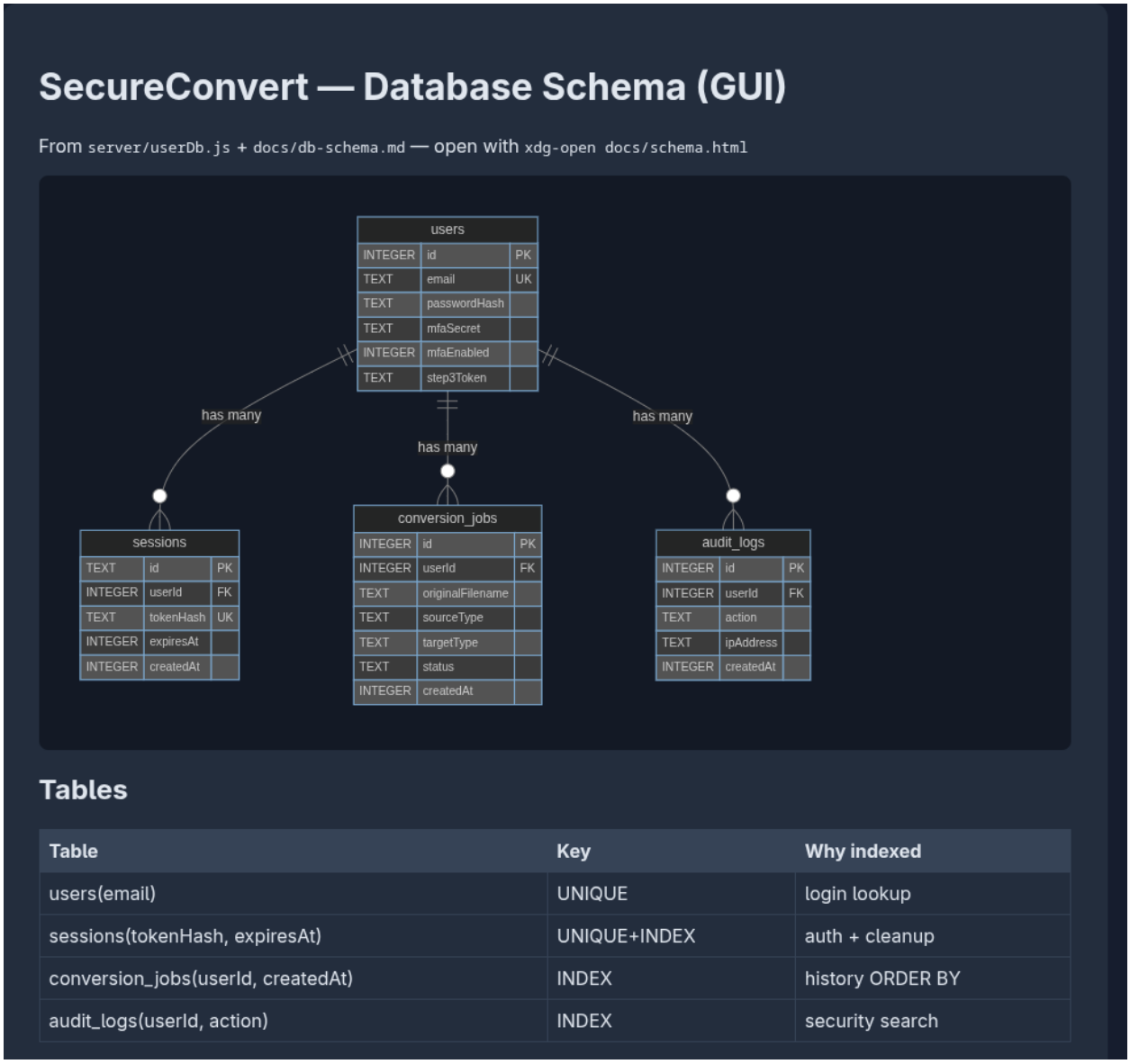

Figure 4b — Earlier dashboard iteration showing authentication and conversion cards side-by-side.

Interaction details that matter

• Drag-and-drop with fallback to file chooser; the drop zone highlights on hover and validates file presence before enabling conversion.

• Format selector that hints at compatibility — for example, explaining which source families can become which targets — so the user learns the model.

• Upload progress via low-level network events, giving feedback for larger files rather than a static spinner.

• History panel that fetches the personal job list after each successful conversion and displays filename, source type, target type and timestamp in reverse chronological order.

• Responsive layout, consistent error placement, safe rendering that escapes dynamic content, and clear sign-out that clears stored tokens.

10 API Design & Request Lifecycle

The API is resource-oriented and stateless except for the session store. Every endpoint returns JSON except for the conversion endpoint, which streams a binary blob with a download disposition. The following overview describes the contract in human terms, not as code.

Endpoint	Method	Purpose	Outcome
Health check	GET	Confirm the service is alive without authentication or rate limiting.	Status, uptime and timestamp.
Supported formats	GET	Advertise which source and target families are supported.	Map of source → allowed targets.
Register	POST	Create an account with email and password; generate second-factor secret.	Success confirmation plus secret, otpauth URL and QR data.
Login step 1	POST	Verify password and issue a temporary token.	Temporary token valid for the next step.
Login step 2	POST	Verify temporary token plus time-based code and issue session token.	Signed session token (one hour).
QR helper	GET	Render a QR code for an otpauth URL as image or data URL.	PNG image or JSON with data URL.
Convert	POST	Authenticated; validate file and format, convert, record job and audit, stream result.	Binary file with content type and download header.
History	GET	Authenticated; list the caller's recent conversion jobs.	Array of jobs ordered by recency.

Lifecycle example — from signup to download

A new user visits the welcome page, follows the signup call to action, provides an email and password, and receives a secret. The secret is added to an authenticator application by scanning the QR code. The user then logs in: first step checks the password and returns a temporary token; second step combines that token with the current six-digit code and returns a session token. The converter page becomes available. The user drags a file onto the zone, selects a target format, and submits. The server checks the session, inspects the file's binary signature, verifies that the target is compatible with the detected source, runs the appropriate in-memory pipeline, records the job and audit, and streams the result. The client triggers a download and refreshes the history, where the new entry appears at the top.

Failure modes are explicit. Missing files, unknown types, incompatible conversions, oversized uploads and invalid names each return a distinct client error with a message that suggests correction. Missing or expired sessions return unauthorised, which the client handles by clearing state and prompting re-authentication. Unexpected internal errors return a generic failure without leaking details, while the server logs the cause for operators.

Validation matrix — what can become what

Detected Source	Allowed Targets	Rationale
Plain text / Markdown	Web page, printable file, plain text	Text is the natural source for document outputs.
Structured data (JSON)	Spreadsheet, web page, printable file, plain text	Structured data can be rendered as documents or flattened to CSV.
Spreadsheet (CSV)	Structured data, web page, plain text	CSV is the inverse of the JSON→CSV path.
Image (PNG/JPEG/WebP/GIF)	PNG / JPEG / WebP	Images are re-encoded between raster formats; document targets are not sensible for pixels.

The matrix is enforced server-side regardless of what the client displays, so a manipulated request cannot bypass compatibility.

11 Testing, Verification & Metrics

Confidence in a security-sensitive system comes from verification at multiple levels: unit tests for isolated logic, integration tests for HTTP behaviour, and operational checks for linting and dependency health. The project includes fifteen automated tests that run with the standard test runner and no external services.

Suite	What Is Tested	Why It Matters
Converter (unit)	Document escaping, PDF header, WebP magic, CSV headers, XSS payload handling.	Ensures pipelines produce correct, safe output without needing the server.
File validator (unit)	PNG signature, JSON parsing, binary rejection.	Validates the trust boundary for file types.
Auth (integration)	Registration, two-step login, bad password rejection, TOTP verification.	Exercises the full identity flow over HTTP with ephemeral databases.
Conversion (integration)	Each supported route plus unsupported format handling, via real multipart uploads.	Proves the authenticated conversion path end-to-end.
Security (integration)	Oversized upload, binary masquerading, markup escaping in outputs.	Demonstrates that hardening is not merely claimed but tested.

How integration tests work without mocks

Integration tests start a real HTTP server on a random port, register a fresh user with a generated email, derive a time-based code from the returned secret, complete both login steps, and then call the conversion and history endpoints with a bearer token. Files are supplied as in-memory buffers, not fixtures on disk, matching the production path. An in-memory database is used so tests do not interfere with development data. This approach catches wiring errors that unit tests cannot — routing, middleware ordering, header handling and streaming — while remaining fast.

Metrics and interpretation

Metric	Result	Interpretation
Tests	15 passing	All pipelines and security properties have at least one automated check.
Lint	0 errors	Consistent style; ignored patterns are explicit.
Audit	0 vulnerabilities	Dependency overrides applied; audit is clean.
Document→PDF	~25 ms	Local, in-process PDF generation for a short document.
Image re-encode	~11 ms	Tiny image (2×2) re-encoded; illustrates sub-second pipelines.
HTTP round-trip	< 1.1 s	Full conversion over loopback including auth and streaming.

Latency is measured on a development machine and will vary, but the order of magnitude is instructive: in-process, memory-only conversion avoids the overhead of spawning workers or touching disk, so the dominant cost is often the library itself. Database queries are logarithmic thanks to indexes on email, user and time, and hashing the session token contains the impact of a database leak.

Manual verification complements automation. A scripted check for binary uploads, injection attempts via crafted emails, and health probes are documented in the repository. Expected outcomes — 400 for bad types, 413 for oversize, unauthorised for missing sessions, and a health response with uptime — are all observable with standard command-line tools.

12 Deployment, Configuration & Operations

The system is designed to be cloned and run with minimal prerequisites, while still respecting production concerns such as secret management, cross-origin control and graceful shutdown.

Prerequisites and quick start

The runtime requirement is a recent Node version and optionally Python for a fallback static server. No separate database server is needed — the SQLite file is created on first run. A typical start involves copying the environment template, generating a strong random secret, installing dependencies for the API and the client, seeding demonstration data, and opening the client. The API and the client run on different ports during development, which is why cross-origin configuration is explicit.

Configuration — environment variables

Variable	Purpose	Typical Value / Guidance
JWT_SECRET	Sign and verify session tokens; must be strong in production.	Random hex of at least 32 characters; generated with a cryptographic RNG.
PORT / HOST	Where the API listens.	5000 / 127.0.0.1 locally; 0.0.0.0 if containerised.
CORS_ORIGINS	Allowed browser origins.	Comma-separated list, e.g. client origin plus any hosted frontend.
RATE_LIMIT_MAX AUTH_RATE_LIMIT_MAX	Global and stricter auth limits per window.	100 and 20 per 15 minutes by default; lower for public deployments.
MAX_UPLOAD_SIZE	Maximum accepted file size.	10 MB by default; aligned with pipeline caps.

Client-side, the API base URL can be overridden via a meta tag or local storage, allowing the static files to be hosted elsewhere without recompilation. Scheduled maintenance such as purging expired sessions is exposed as a function that can be invoked on startup or via a periodic job.

Operational hygiene

- **Health check.** An unauthenticated endpoint returns status, uptime and timestamp, suitable for load-balancer probes and uptime monitors.
- **Stateless scaling.** Because only metadata is persisted and sessions are in the database, multiple API processes could share the same SQLite file in WAL mode for read concurrency, though a single instance suffices for the portfolio scope.
- **Graceful shutdown.** The server handles termination signals by closing the listener before exiting, avoiding in-flight request truncation.
- **Security headers.** Helmet removes the powered-by header and sets hardening headers; CORS is configured from environment rather than hardcoded.
- **Observability.** Conversion attempts log the filename, detected type and user identifier server-side; audit logs provide a queryable history.

Seeding and inspecting the database

A seeding script creates three demonstration users and seven conversion jobs without requiring a database client, by using the same data-access layer as the application. Inspection can be done with the SQLite command line or a graphical browser, and the printable schema page documents example queries including a join that counts conversions per user and an aggregation by target format. These are designed for live demonstration during a defence: a few commands and the relational model becomes tangible.

13 Challenges Encountered & Lessons Learned

Building a small but realistic system surfaces the same friction that larger projects face — dependency drift, environment differences and the tension between usability and safety. Three challenges were particularly instructive.

Challenge	Symptom	Resolution	Lesson
Static server auto-open	The previous file-serving command failed on newer runtimes due to an unknown option.	Replaced with a tiny launcher that starts the server and opens the browser via platform-specific commands, plus a Python fallback.	Wrap tooling rather than relying on CLI flags that may change across versions.
Dependency audit	A transitive dependency introduced moderate advisories.	Pinned the affected package via overrides and re-installed, achieving a clean audit.	Overrides are a legitimate, auditable fix; clean audits are worth the extra configuration.

Challenge	Symptom	Resolution	Lesson
Database WAL artefacts	After enabling write-ahead logging, auxiliary files appeared and a stale state caused corruption errors.	Ignored auxiliary files in version control, documented clean-up and re-seeding, and grouped migrations.	Document the files that WAL creates and provide a one-command reset for demos.

Broader lessons

- Whitelist, do not blacklist: column names in updates are validated against an allowed set, so even a crafted payload cannot choose an arbitrary column.
- Hash what you store: keeping only a hash of the bearer token means a stolen database is not a stolen session store.
- Validate what you receive, not what you display: compatibility between source and target is re-checked server-side even though the client hints at it.
- Make the audit trail independent: retaining audit logs after user deletion preserves accountability without retaining personal data indefinitely.
- Keep the demo path trivial: seeding and schema inspection must work without installing extra tools, otherwise an examiner will not see them.
- Treat limits as features: size, pixel and row caps are documented and tested, not hidden, because they are part of the security model.

14 Future Improvements & Roadmap

The project is complete for its academic scope but leaves clear, incremental extensions that would be natural next steps for a production evolution or for further study.

Area	Proposed Enhancement	Benefit
Authentication	Add passkeys / WebAuthn as an optional third factor; support recovery codes.	Phishing-resistant authentication and better account recovery.
Storage	Offer client-side encrypted storage option where the server never sees plaintext.	Even stronger privacy for the most sensitive documents.
History & Audit	Paginated, searchable history and a dedicated audit endpoint with filters.	Usability for power users and better forensics for operators.
Rate limiting	Per-user limits keyed by session, not just per address.	Fairer under shared IPs and more precise abuse control.
Performance	ETag / conditional responses for repeated conversions of identical content.	Avoids redundant work for identical inputs.
Operations	Continuous integration running tests, lint and audit on every push; container image.	Automated quality gates and one-command deployment.
Formats	Additional pipelines (e.g. more document and data formats) behind the same validation model.	Broader utility without weakening trust.

Each item is deliberately scoped so that it can be added without re-architecting the core: authentication extensions fit the existing session model, history pagination builds on the indexed queries, and new pipelines reuse the magic-byte and compatibility framework.

15 Conclusion

SecureConvert demonstrates that a file converter can be both easy to use and careful with trust. By processing entirely in memory, validating by content rather than by name, requiring two-factor authentication and recording an

auditable history in a well-designed relational store, it addresses the weaknesses that make typical converters risky. The implementation is small enough to be understood in a single sitting — a static client, an Express API and a SQLite database — yet it touches the concerns that distinguish a student exercise from a defensible system: security headers, rate limits, cryptographic handling, binary inspection, transactional integrity and verifiable tests.

For the University of Messina, the project satisfies both the web-security and database briefs: it shows ephemeral handling instead of careless persistence, it models identity, sessions, jobs and audits as relations with foreign keys, unique constraints, indexes, migrations and transactions, and it does so with parameterised queries and whitelisted updates. For a portfolio, it shows that the author can make and justify technology choices, handle errors gracefully, measure and explain performance, and document a system so that another person can run and demo it without assistance.

The most important outcome is not the set of supported formats but the habit of treating every input as untrusted and every claim as needing evidence. That habit — visible in the magic-byte checks, the hashed sessions, the server-side compatibility enforcement and the audit log — is transferable to any system that accepts user content. The report, together with the live system, the schema page and the screenshots in the assets folder, provides the evidence that the habit has been practised.

Prepared for Prof. Armando Ruggeri
Università degli Studi di Messina — Dipartimento di Ingegneria

Student: **Kheizaran Nazari** — Matricola 557463
kheizarannazarikhakeshoori@gmail.com
github.com/kheizaran-nazari-khakeshoori/zero-trust-ephemeral-converter

September 2026 — Messina, Italy — This report contains no source code listings; it describes behaviour, rationale and verification.

Appendices

Appendix A — Glossary

Term	Meaning in This Project
Zero-trust	Treat every file, request and stored value as potentially hostile; verify at each layer.
Ephemeral	Exists only in volatile memory for the duration of the request; not written to disk.
Magic bytes	Binary signatures at known offsets that identify a file's true type regardless of its name.
TOTP	Time-based one-time password — a six-digit code that changes every ~30 seconds, generated from a shared secret.
JWT / jti	JSON Web Token — a signed, short-lived token; jti is its unique identifier stored as a session row.
WAL	Write-ahead logging — SQLite mode that allows concurrent readers and a writer with good durability.
Helmet	Express middleware that sets hardening HTTP headers (HSTS, frame blocking, etc.).
Multer (memory)	Upload handler that keeps files as in-memory buffers instead of temporary files.
Parameterized query	SQL with placeholders (?) so that user input never becomes part of the statement text.

Appendix B — Repository Structure

The repository is organised for clarity; an examiner can locate any concern in seconds.

Path	What It Contains
assets/	Screenshots for the report and README (this folder).
client/index.html client/converter.html client/login.html / signup.html	Welcome, converter and authentication pages (static).
client/css/style.css	Shared styling — responsive, dark theme.
client/js/*	Page logic: auth flows, drag-and-drop, progress, history rendering.
server/server.js	Express application: middleware, routes, conversion endpoint, history, health.
server/userDb.js	SQLite + migrations, indexes, transactions, audit helpers.
server/authRoutes.js server/authMiddleware.js	Registration, two-step login, QR helper, bearer verification.
server/converter.js server/fileValidator.js	In-memory pipelines and binary-signature inspection.
server/config.js server/inputValidation.js	Environment-driven config and input whitelisting.
docs/db-schema.md docs/schema.html	Printable schema documentation and diagram.
docs/Report_*.pdf	This report and prior versions.
demo-files/	Sample Markdown and JSON for quick manual tests.

Appendix C — How to Demo in 90 Seconds

The following steps are designed for a live defence where time is short and reliability matters. No database client is required beyond what Node provides.

1. Start the API and then the client; confirm the health endpoint returns status 'ok'.

2. Show the welcome page and navigate to signup; create an account and display the QR code and secret (explain that it works with any authenticator).

3. Log in in two steps — password first, then the current six-digit code — and show that the converter becomes available only after the second step.

4. Drag a Markdown file, select web page as target, convert, and trigger the download; repeat with JSON→CSV and an image re-encode to show all families.

5. Refresh the history panel and show the new rows with timestamps; then open the schema page or run the seeding script to illustrate the four tables and indexes.

6. Optionally demonstrate rejection: attempt a binary file masquerading as an image, and a tampered filename — both return clear validation errors.

All steps use the files in assets and demo-files, so the examiner sees the same content that is documented here.

List of Tables

References

[1] OWASP — File Upload Cheat Sheet: trust content, not extension; enforce size and type; store outside web root (here, not at all).

[2] Helmet.js documentation: security headers (HSTS, X-Content-Type-Options, X-Frame-Options).

[3] Speakeasy / RFC 6238 — Time-based One-Time Password (TOTP) standard; window parameter tolerates clock drift.

[4] bcrypt — adaptive password hashing with salt and configurable cost; cost 12 as used here.

[5] Sharp documentation — failOnError and limitInputPixels for safe image decoding; quality-controlled encoding.

[6] PDFKit documentation — streaming PDF generation without temporary files.

[7] SQLite documentation — foreign keys, WAL mode, EXPLAIN QUERY PLAN, user_version for migrations.

[8] Express.js, express-rate-limit, Multer — memoryStorage for ephemeral uploads.

[9] Mozilla Developer Network — CORS, Content-Disposition: attachment, download via Blob.

End of report — SecureConvert, University of Messina, September 2026. For the live system, see the repository README for the exact commands to start the API and client, seed the database and open the printable schema.